3. Distributed Shared Data

Concurrence, Parallelism and Distributed Systems (CPDS)
Facultat d'Informàtica de Barcelona (FIB)
Universitat Politècnica de Catalunya (UPC)
2025/2026

FIB

3.A. Distributed Transactions

Concurrence, Parallelism and Distributed Systems (CPDS)
Facultat d'Informàtica de Barcelona (FIB)
Universitat Politècnica de Catalunya (UPC)
2025/2026

FIB

Contents

- **Introduction**
- Problems with concurrent transactions
- Concurrency control
- Distributed transactions

Introduction

- Provide atomicity and isolation to a group of operations at a server in the presence of multiple clients and process crashes
- Properties - ACID
  - Atomicity: either all operations are completed or none of them is executed
  - Consistency: takes the system from one consistent state to another (this is an application concern)
  - Isolation: updates of one transaction are not visible to other transactions until it commits
  - Durability: persistent once completed successfully

Contents

- Introduction

- **Problems with concurrent transactions**

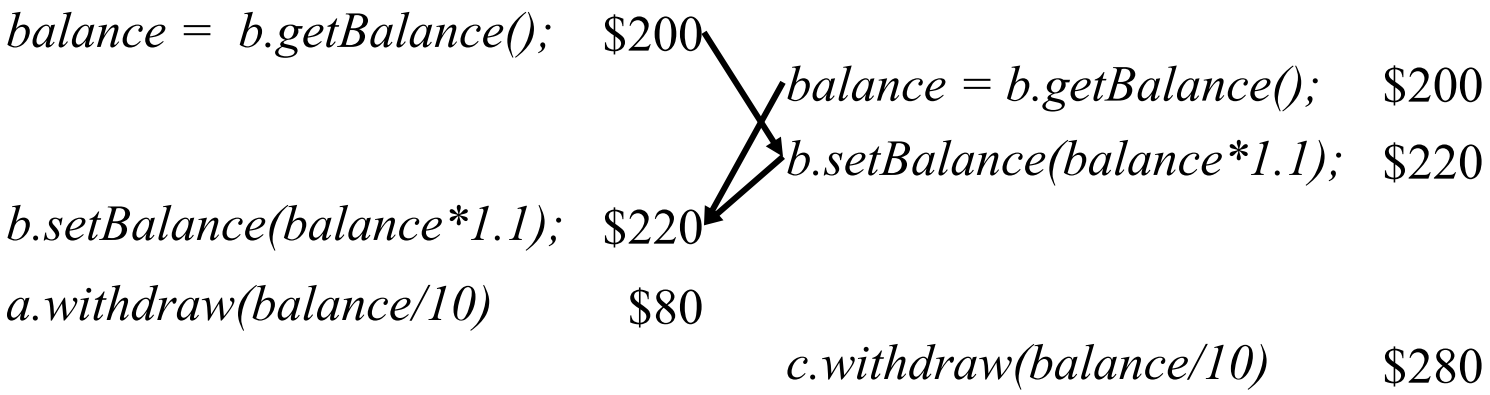
- Concurrency control

- Distributed transactions

Lost update problem

Transaction T:	Transaction U:
balance = b.getBalance(); *b.setBalance(balance*1.1);* *a.withdraw(balance/10)*	*balance = b.getBalance();* *b.setBalance(balance*1.1);* *c.withdraw(balance/10)*

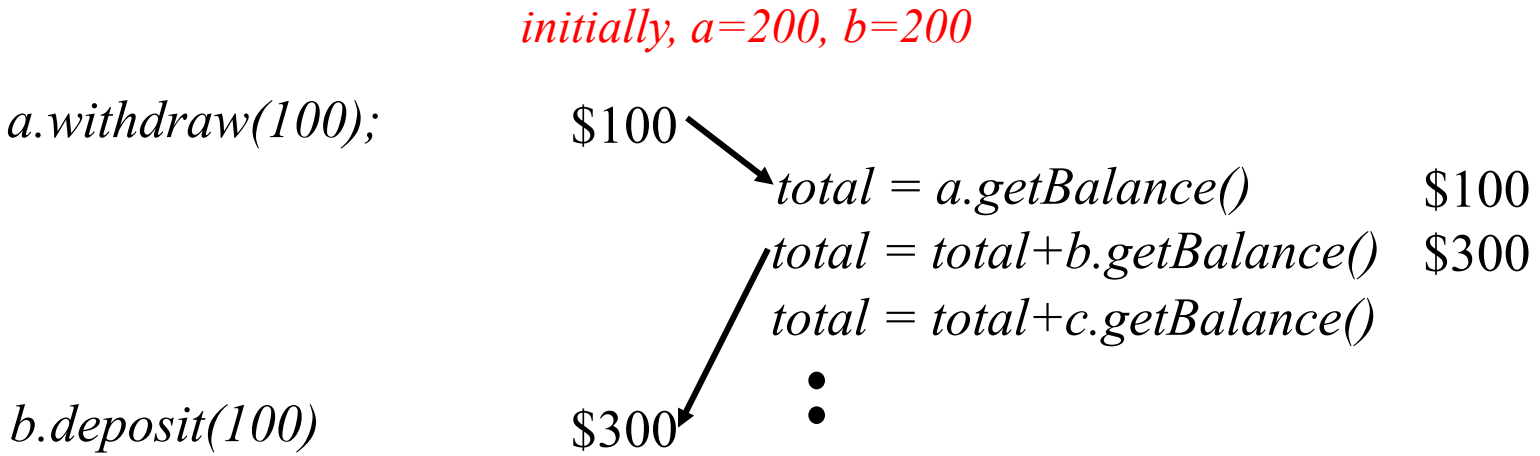
initially, a=100, b=200, c=300



- U's update on 'b' is lost as T overwrites 'b' without seeing it

Inconsistent retrievals problem

Transaction V:	Transaction W:
a.withdraw(100) *b.deposit(100)*	*aBranch.branchTotal()*



- W's retrievals are inconsistent because V has only performed the withdrawal at the time the sum is calculated

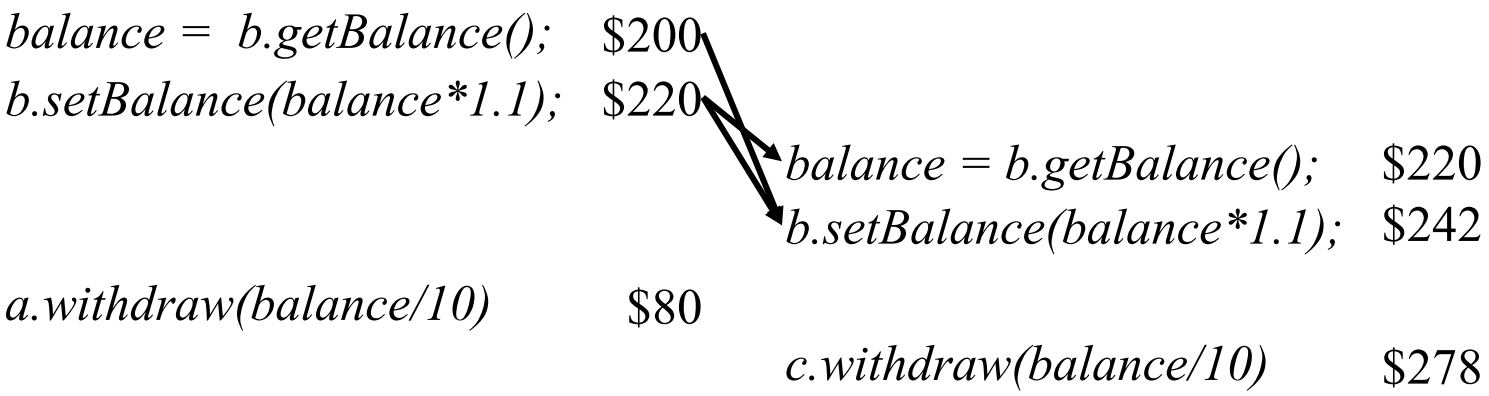
Concurrency control

- Allow concurrent transactions to be executed correctly, i.e. preserving **serial equivalence**
  - The outcome is the same as if the transactions had been performed one at a time in some order
- Two transactions are serially equivalent if all pairs of conflicting operations are executed in the same order at all the shared objects
  - i.e. the same transaction operates always first
  - Two operations are conflicting if their effect depends on the order in which they are executed
    - read-write, write-write

Lost update revisited

Transaction T:	Transaction U:
balance = b.getBalance();	*balance = b.getBalance();*
*b.setBalance(balance*1.1);*	*b.setBalance(balance*1.1);*
a.withdraw(balance/10)	*c.withdraw(balance/10)*

initially, a=100, b=200, c=300



- A serially equivalent interleaving of T and U

Inconsistent retrievals revisited

Transaction V:	Transaction W:
a.withdraw(100) *b.deposit(100)*	*aBranch.branchTotal()*

initially, a=200, b=200

a.withdraw(100);	$100	*total = a.getBalance()*	$100
b.deposit(100)	$300	*total = total+b.getBalance()*	$400
		total = total+c.getBalance()	
		⋮	

- A serially equivalent interleaving of V and W

Dirty read problem

Transaction T:	Transaction U:
a.getBalance() *a.setBalance(balance + 10)*	*a.getBalance()* *a.setBalance(balance + 20)*

balance = a.getBalance()	$100
a.setBalance(balance + 10)	$110
	balance = a.getBalance() $110
	a.setBalance(balance + 20) $130
	commit transaction
abort transaction	

- U has seen a value that never existed

Recoverability from aborts

a) OPTION A: Delay the commit operation until earlier transactions that wrote the same objects have committed/aborted
   - e.g. U must delay its commit until T commits
   - If earlier transaction aborts, delayed transaction must also abort
     - e.g. if T aborts, then U must abort as well
   - This may cause **cascading aborts**

b) OPTION B: Delay any read operation until earlier transactions that wrote the same object have committed/aborted

Premature write problem

Transaction T:		Transaction U:	
a.setBalance(105)		*a.setBalance(110)*	
	$100		
a.setBalance(105)	$105		
		a.setBalance(110)	$110
		commit transaction	
abort transaction			

- When T aborts, 'a' is reverted to an incorrect value
- Delay any write operation until earlier transactions that wrote the same object have committed/aborted

Recoverability from aborts

- To avoid dirty reads and premature writes, we require **strict execution** of transactions
  - Both read and write operations on an object must be delayed until earlier transactions that wrote that object have been aborted or committed
- Use tentative versions to allow recoverability
  - All the updates performed during a transaction are done in tentative versions of the objects
  - Read from tentative versions if possible
  - Updates are transferred to the objects only when a transaction commits, and discarded if it aborts

Contents

- Introduction
- Problems with concurrent transactions
- **Concurrency control**
- Distributed transactions

Concurrency control

- Schedule concurrent transactions so that they execute preserving **serial equivalence**
  - Allow as much concurrency as possible
- Algorithms can use different options to schedule an operation within a transaction
  - A) Execute it, B) Delay it, C) Abort it

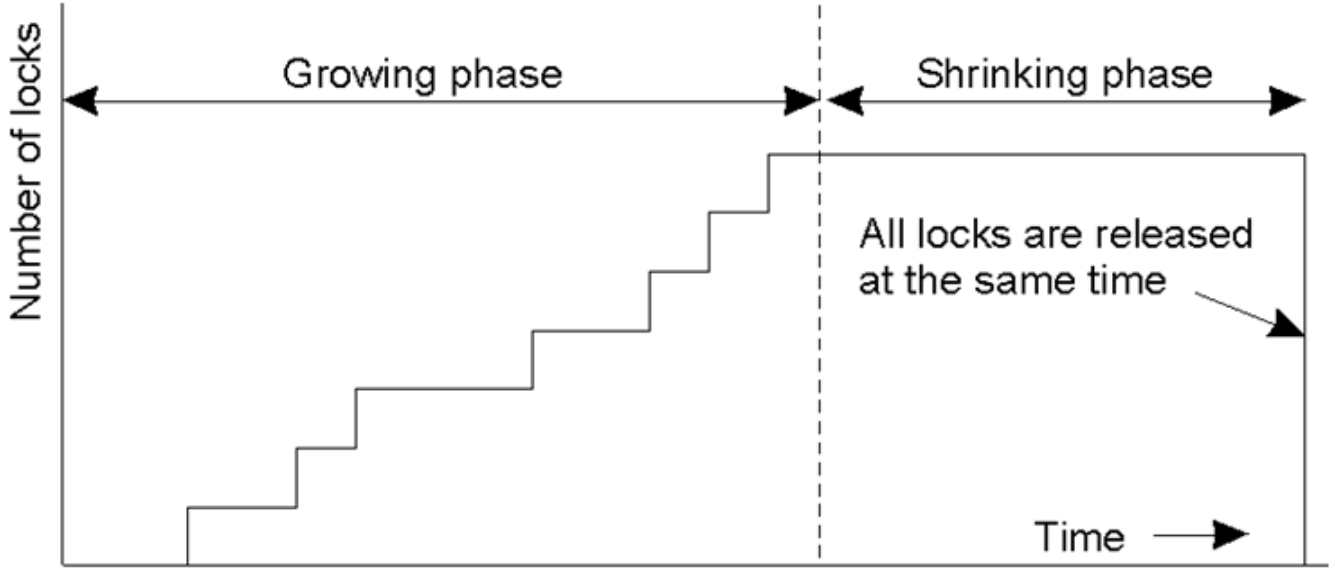
1. Strict two-phase locking
2. Timestamp ordering
3. Optimistic concurrency control

Contents

- Introduction
- Problems with concurrent transactions
- **Concurrency control**
  - **Strict two-phase locking**
  - Timestamp ordering
  - Optimistic concurrency control
- Distributed transactions

Strict two-phase locking

- Serialize access to the objects using locks



- A transaction is not allowed new locks after it has released a lock to get serial equivalence
- Locks must be held until the transaction commits or aborts to get strict executions

Strict two-phase locking

Transaction T:		Transaction U:	
balance = b.getBalance() *b.setBalance(bal*1.1)* *a.withdraw(bal/10)*		*balance = b.getBalance()* *b.setBalance(bal*1.1)* *c.withdraw(bal/10)*	
Operations	**Locks**	**Operations**	**Locks**
openTransaction			
bal = b.getBalance()	lock B		
*b.setBalance(bal*1.1)*		*openTransaction*	
a.withdraw(bal/10)	lock A	*bal = b.getBalance()*	wait for T's lock on B
closeTransaction	unlock A,B	...	lock B
		*b.setBalance(bal*1.1)*	
		c.withdraw(bal/10)	lock C
		closeTransaction	unlock B,C

Strict two-phase locking

- How to increase the concurrency of locking?
  - Use different locks for read and write access
  - 'Multiple readers/single writer' scheme
    - Multiple transactions can take read locks but only if the write lock is not taken
    - Only one transaction can take a write lock but only if the read and write locks are not taken
    - Read locks can be promoted to write locks (if not shared)

for one object		*lock requested*	
		read	*write*
lock already set	*none*	OK	OK
	read	OK	wait
	write	wait	wait

Strict two-phase locking

1. When a read or write operation accesses an object within a transaction:
   a) If the object is not already locked, it is locked and the operation proceeds
   b) If the object has a conflicting lock set by another transaction, the transaction must wait until it is unlocked
   c) If the object has a non-conflicting lock set by another transaction, the lock is shared and the operation proceeds
   d) If the object has already been locked in the same transaction, the lock will be promoted if necessary and the operation proceeds (where promotion is prevented by a conflicting lock, rule b is used)
2. When a transaction commits or aborts, the server unlocks all objects it locked for the transaction

Strict two-phase locking

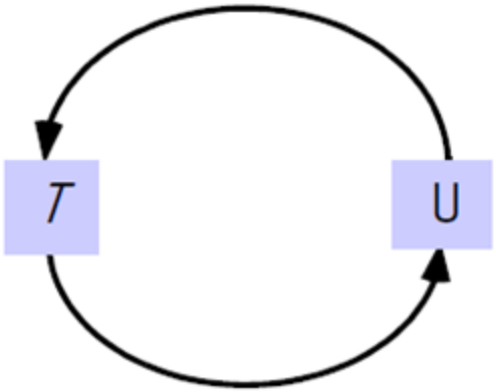
↓ Deadlocks can occur!

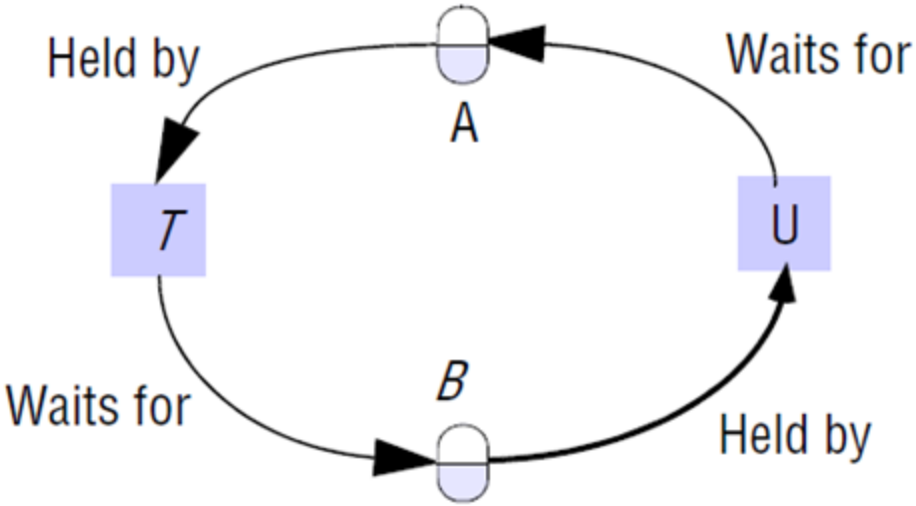
Transaction T		Transaction U	
Operations	**Locks**	**Operations**	**Locks**
a.deposit(100)	write lock A		
		b.deposit(200)	write lock B
b.withdraw(100)	wait for U's lock on B		
		a.withdraw(200)	wait for T's lock on A

- Deadlocks could be prevented taking locks all at once when transaction starts but this reduces concurrency

Deadlock detection

- A **wait-for graph** can be used to represent the waiting relations between transactions
- Check for **cycles** to detect deadlocks
  a) Each time an edge is added (i.e. a lock is taken)
  b) Less frequently to avoid unnecessary overhead





Deadlock resolution

a) Abort one transaction involved in the cycle
   - Which transaction should be aborted?
     - Consider the age of the transaction and the number of cycles in which it is involved

b) Lock timeouts
   - A taken lock is made **vulnerable** after a timeout
   - Lock is broken if any other transaction is waiting
   - Transaction with the broken lock is aborted
   - ↓ Unnecessary aborts: sometimes, other transactions are waiting, but there is no deadlock
   - ↓ How to choose an adequate timeout value?

Contents

- Introduction
- Problems with concurrent transactions
- **Concurrency control**
  - Strict two-phase locking
  - **Timestamp ordering**
  - Optimistic concurrency control
- Distributed transactions

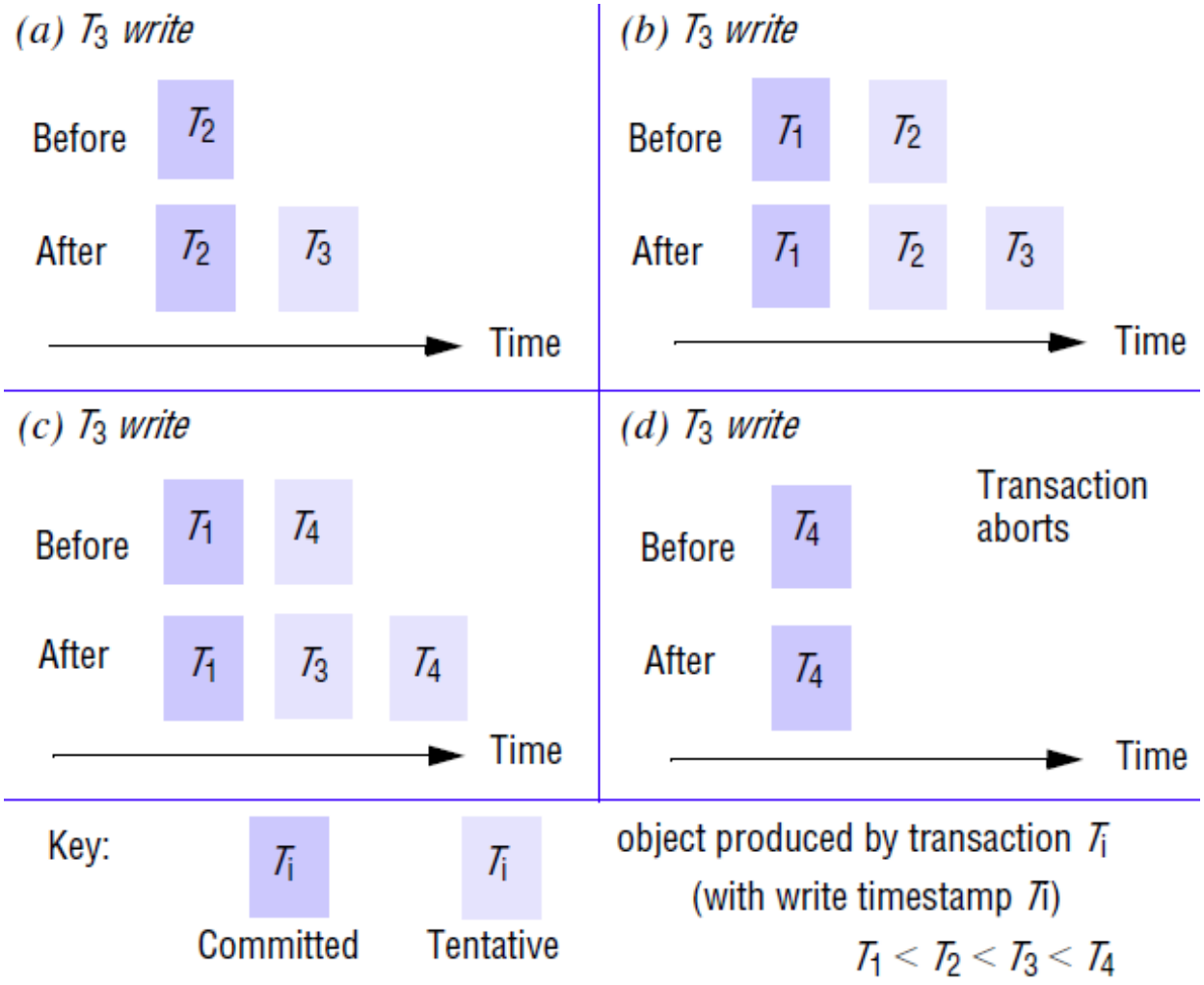
Timestamp ordering

- Each transaction has a unique timestamp
  - Defines its position in the sequence of transactions
  - Assigned when the transaction starts
- Each object x keeps:
  1. Write timestamp for its committed value: wts(x)
  2. Read timestamps, denoted by the highest: rts(x)
  3. Pending read and tentative write operations with their corresponding timestamps
- Operations are validated when performed by comparing object and transaction timestamps

Timestamp ordering

- Transaction T performs **write** operation on x
  - Valid only if object x was last read and written by earlier transactions
    - (ts(T) ≥ rts(x)) AND (ts(T) > wts(x))
  - Add to the tentative writes a new version of the object with a write timestamp set to ts(T)
    - Overwrite version if it already exists
    - Tentative versions are kept ordered by their timestamps
  - If write is not valid, transaction T is aborted
    - A transaction with a later timestamp has already read or written the object

Timestamp ordering



T_3 >= rts

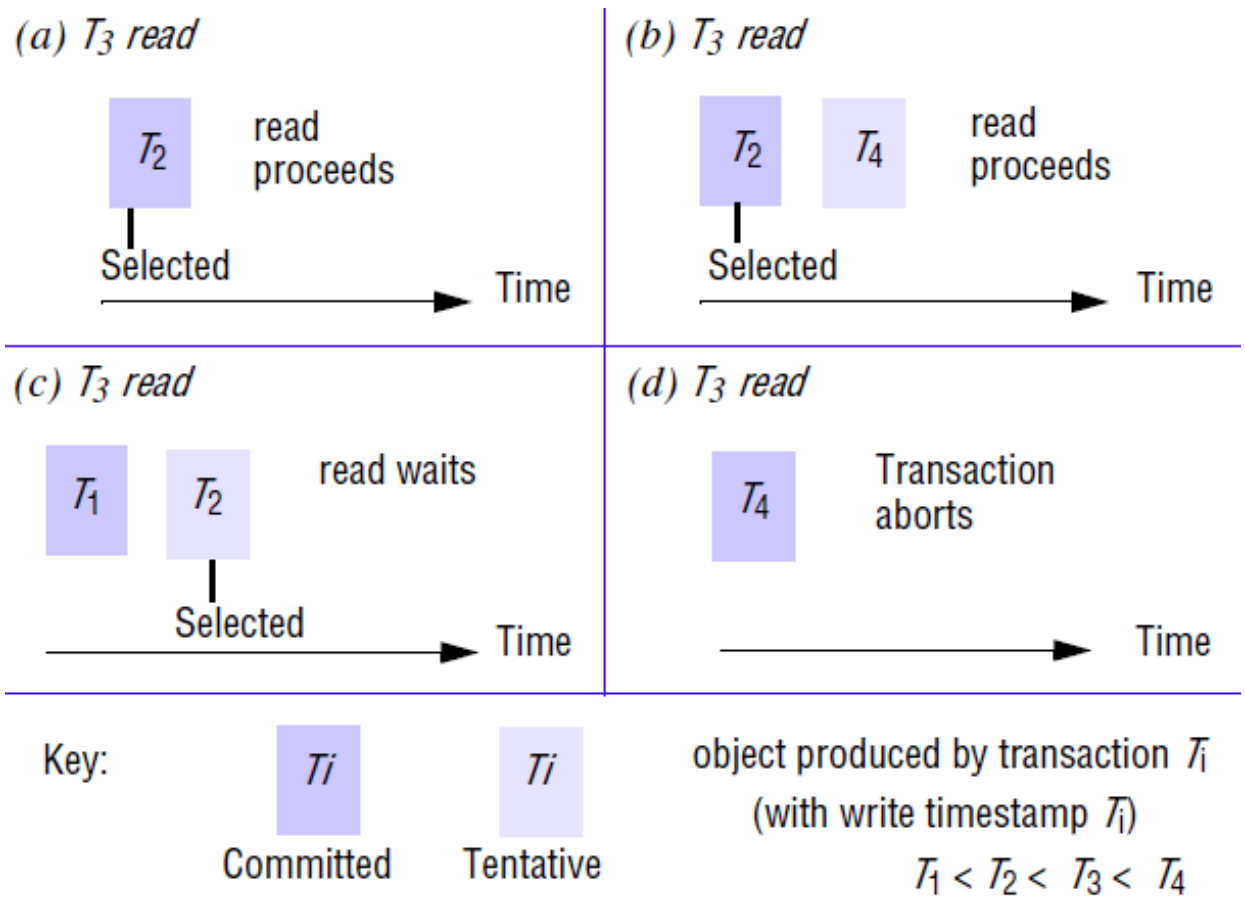
(a)-(c) T_3 > wts

(d) T_3 < wts

Timestamp ordering

- Transaction T performs **read** operation on x
  - Valid only if object x was last written by an earlier transaction
    - ts(T) > wts(x)
  - Must read the version with the highest write timestamp lower than the transaction one
    - If this is still tentative, read stays pending until the earlier transaction completes
    - Otherwise, read committed version and set rts(x) to max{rts(x), ts(T)}
    - If T has already written the object, this will be used
  - If read is not valid, transaction T is aborted

Timestamp ordering



(a)-(c) T_3 > wts

(a)-(b) read proceeds with committed version

(c) read directed to a tentative version: T_3 must wait

(d) T_3 < wts

Timestamp ordering

- Transaction T performs **commit** operation
  - For each object x written by T, replace committed version with tentative one and set wts(x) to ts(T)
    - If x has any tentative write or pending read or commit operation with lower timestamp than T, commit stays pending to wait for those earlier operations to complete
    - Otherwise, commit proceeds and also resumes pending read and commit operations waiting for T
- Transaction T performs **abort** operation
  - Discard all tentative versions written by T
  - Update rts(x) of all objects read by T (if needed)

Contents

- Introduction
- Problems with concurrent transactions
- **Concurrency control**
  - Strict two-phase locking
  - Timestamp ordering
  - **Optimistic concurrency control**
- Distributed transactions

Optimistic concurrency control

- Based on the premise that most transactions do not conflict
- Transactions proceed in three phases:
  1. Working phase: Operations proceed as there are no conflicts, but writing to a private workspace
  2. Validation phase: On transaction commit, check if it has conflicted with concurrent transactions
  3. Update phase: Tentative versions of any write operations are committed to the database

Optimistic concurrency control

1. Working phase
   - Transaction keeps a tentative version of each object that it updates
     - This allows the transaction to abort with no effect on the objects
   - Transaction keeps also the read and write set
     - read set records the objects read by the transaction
     - write set records the objects written by the transaction
   - Read operations directed to the tentative version if exists, otherwise to the committed one
   - Write operations only onto the tentative version

Optimistic concurrency control

2. Validation phase
   - A transaction is given a sequence number when entering the validation phase, but real assignment is postponed until successful validation and update
   - On transaction T_v commit, check for conflicts with **overlapping** T_i transactions (those not yet committed at the start of T_v)
   - If the transaction is valid, it is allowed to commit
   - Otherwise, abort the transaction
     - Or perform some conflict resolution
       - e.g. Amazon Dynamo, Wikipedia, file synchronizers such as Dropbox or SVN

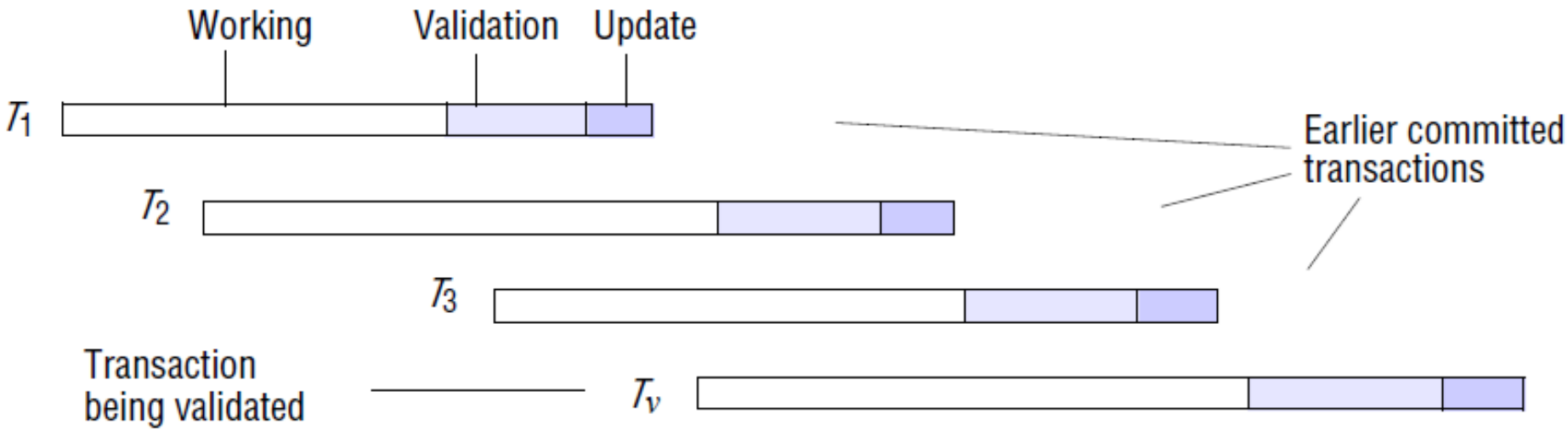
Optimistic concurrency control

A. Backward validation
   - Validate a transaction T_v by comparing its read set with the write set of (committed) transactions T_i with sequence number $T_{start} < T_i \leq T_{end}$
     - T_{start} is the biggest sequence number assigned when transaction T_v enters the working phase
     - T_{end} is the biggest sequence number assigned when transaction T_v enters the validation phase
   - If a conflict is detected, abort **this** transaction

↓ All the write sets of overlapping transactions have to be kept until their last concurrent transaction has finished

Optimistic concurrency control

A. Backward validation



- The read set of T_v must be compared with the write sets of T_2 and T_3

Optimistic concurrency control

B. Forward validation

- Validate a transaction T_v by comparing its write set with the read set of overlapping active (uncommitted) transactions

↑ Flexible in conflict resolution

a) Abort the transaction being validated

b) Abort the conflicting active transactions

c) Let the conflicting transactions finish and try again

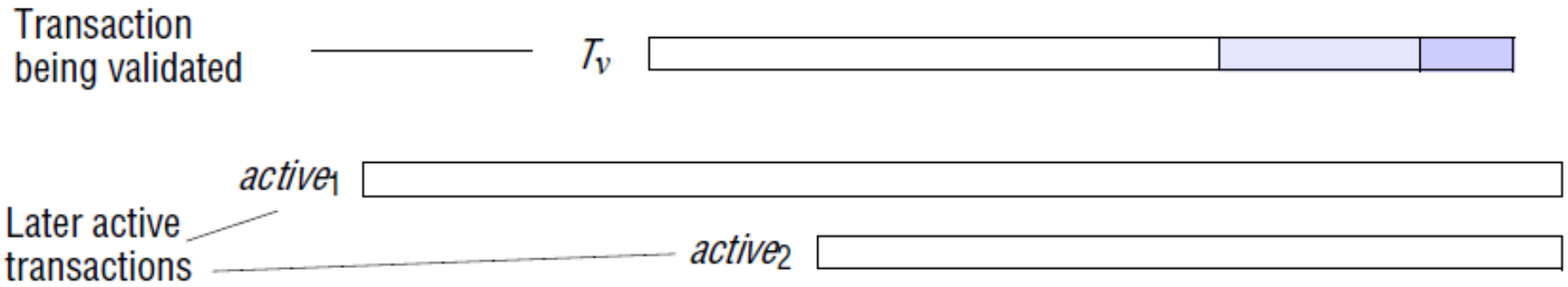
- One of them might abort …

↓ Dynamic read sets must be consistently checked

- e.g. block reads of active transactions on contended objects while doing the validation and update phases

Optimistic concurrency control

B. Forward validation



- The write set of transaction T_v must be compared with the read sets of the transactions $active_1$ and $active_2$

SEMINAR PREPARATION – Opty

- **[Kung81]** Kung, H.T., Robinson, J.T., *On Optimistic Methods for Concurrency Control*, ACM Transactions on Database Systems, Vol. 6, No. 2, pp. 213-226, June 1981
- **[Harder84]** Harder, T., *Observations on Optimistic Concurrency Control Schemes*, Information Systems, Vol. 9, No. 2, pp. 111-120, November 1984

Comparing concurrency control methods

Two-phase Locking

- Pessimistic: conflicts detected (& serialization order decided) as objects are accessed and solved delaying the transaction
- Incurs overhead even if there is no conflict
- Risk of deadlocks: its prevention reduces concurrency
- Better when operations are mainly updates

Timestamp Ordering

- Pessimistic: conflicts detected as objects are accessed and solved aborting the transaction
- Serialization order decided when transaction starts
- Better when operations are mainly reads

Optimistic Conc Ctrl

- Optimistic: transaction proceeds normally, conflicts detected (& serialization order decided) when it tries to commit and solved aborting the transaction
- Better when there are few conflicts

Contents

- Introduction
- Problems with concurrent transactions
- Concurrency control
- **Distributed transactions**

Distributed transactions

- A transaction can access objects managed by multiple servers
  - For each transaction, one server acts as coordinator, being responsible of opening and closing (commit/abort) the transaction
  - Each of the servers accessed by a transaction is a participant

➢ All servers must commit the transaction or all of them must abort → **distributed commit**

➢ How to schedule operations to guarantee serial equivalence? → **concurrency control**

Contents

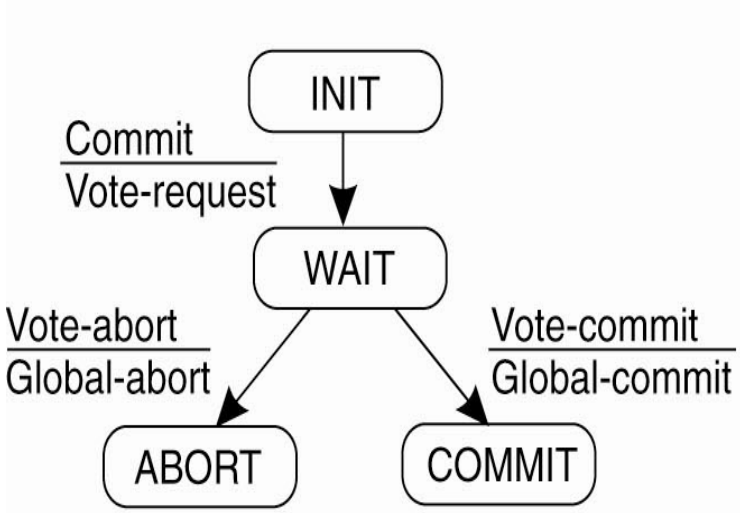
- Introduction
- Problems with concurrent transactions
- Concurrency control
- **Distributed transactions**
  - **Distributed commit**
  - Concurrency control

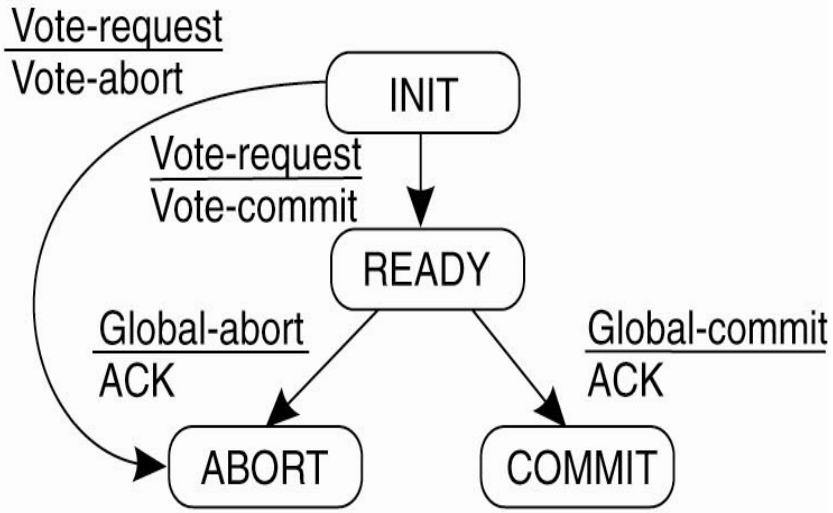
Distributed commit

- To ensure atomicity, all the servers accessed by a transaction must agree on the final outcome of the execution (commit/abort)
- One-phase commit is not feasible
  - Coordinator tells all participants to commit
  - If a participant needs to abort (due to concurrency control issues), it cannot inform the coordinator
- Two-phase commit protocol (2PC) is used
  - It is a consensus protocol, but …
    - All participants must vote and reach the same decision
    - If any participant votes to abort, all must abort

Two-phase commit protocol (2PC)

1. Coordinator sends Vote-request to all the participants (including itself)
2. Each participant replies Vote-commit if can commit locally, if not, it replies Vote-abort and directly aborts
3. Coordinator collects all the votes (including its own). If **everyone** voted to commit, it sends Global-commit, otherwise, Global-abort
4. Participants COMMIT or ABORT according to the message and optionally send ACK when done





Two-phase commit protocol (2PC)

- Designed for asynchronous systems in which servers may crash and messages may be lost
  - Remember that consensus cannot be guaranteed under those conditions, but can be achieved with some probability

  ⇒ Mask crash failures by recovering the state of the crashed process from permanent storage
    - Once a participant has voted to commit, it must ensure that it will eventually be able to carry out the commit

  ⇒ Use timeouts and decide (if possible) when processes block due to lost messages or crashes
    - In case of any doubt, revert to a fail-safe configuration

Two-phase commit protocol (2PC)

A. Coordinator blocked in WAIT
   - Ask the participant to send again the vote
   - If there is no response, decide ABORT

B. Participant blocked in READY
   a) Ask the coordinator to resend the global decision
   b) Query other participants about their state
      - If COMMIT→ COMMIT; If ABORT or INIT→ ABORT
   - If no response or all participants are READY, they cannot decide until the coordinator recovers

⇒ It is a blocking protocol (it is safe, but not live)

Distributed commit

- Despite its blocking nature on coordinator failure, 2PC is still a very popular consensus protocol, due to its low message complexity
- Three-phase commit protocol also exists
  - Avoids the blocking problem, but at the cost of adding one more step, resulting in higher latencies
  - It also falls short upon network partitions, so it is not applied often in practice
- Alternatively, 2PC can sit on top of Paxos

  $\Rightarrow$ Paxos Commit

Contents

- Introduction
- Problems with concurrent transactions
- Concurrency control
- **Distributed transactions**
  - Distributed commit
  - **Concurrency control**

Concurrency control

- Schedule concurrent transactions so that they execute preserving serial equivalence
  a) Each server is responsible for concurrency control of its own objects
  b) All servers must agree on the same order of transactions to preserve serial equivalence
     - If transaction T is before U in their access to objects in one server then all servers should have T before U

1. Strict two-phase locking
2. Timestamp ordering
3. Optimistic concurrency control

Strict two-phase locking

- For each object, locks are maintained locally in its server
- Locks are held until transaction is committed or aborted **at all servers** involved

Strict two-phase locking

↓ Distributed deadlocks can happen!

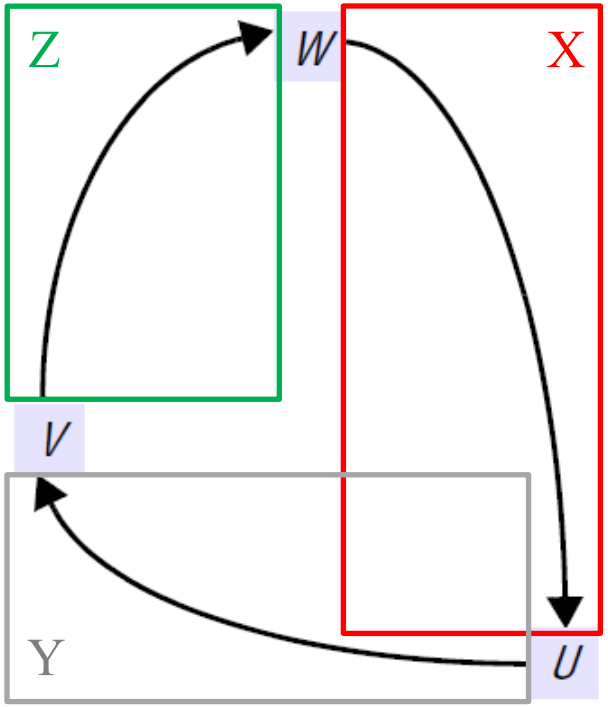
objects A,B, and C are managed by servers X,Y, and Z, respectively

Transaction U		Transaction V		Transaction W	
Operations	**Locks**	**Operations**	**Locks**	**Operations**	**Locks**
		b.deposit(10)	lock B at Y		
a.deposit(20)	lock A at X				
				c.deposit(30)	lock C at Z
b.withdraw(30)	wait at Y for V's lock on B				
		c.withdraw(20)	wait at Z for W's lock on C		
				a.withdraw(20)	wait at X for U's lock on A

Distributed deadlock

A. Centralized detection

- A server builds a global wait-for graph from the local graphs
  1. Looks for cycles in the global wait-for graph
  2. Tells servers which transaction to abort when detects a cycle



↓ Poor scalability & fault tolerance

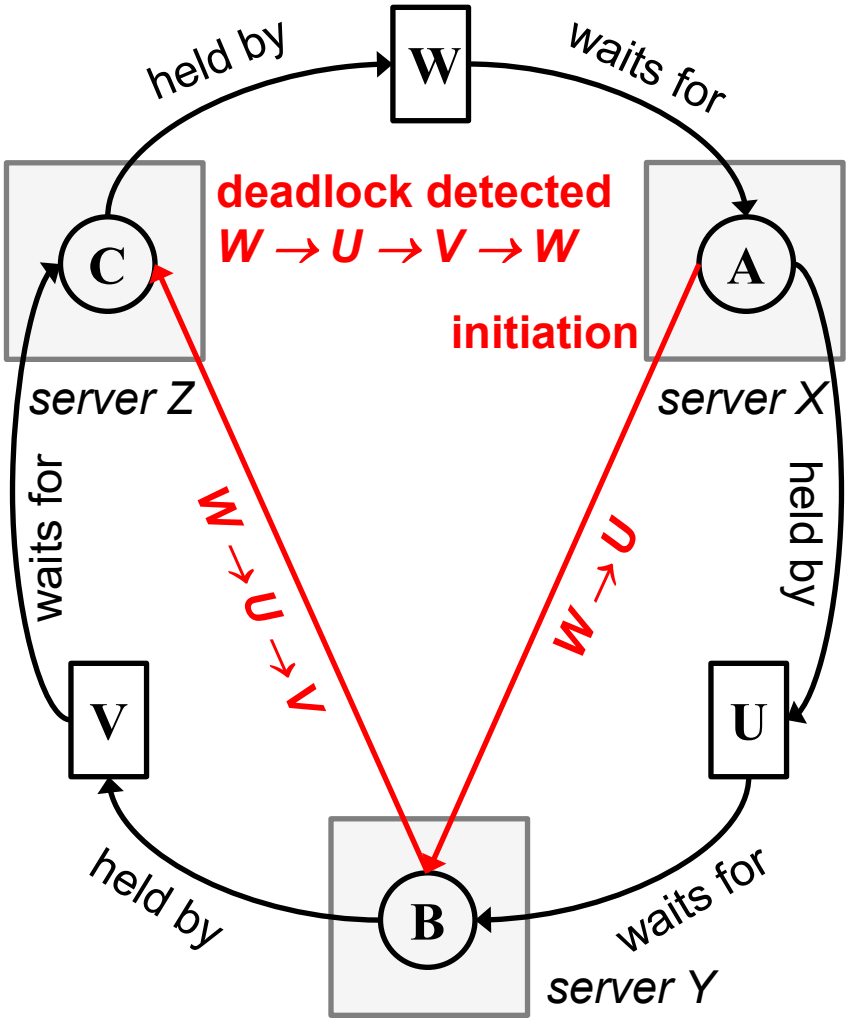
↓ How often should we collect local wait-for graphs?

- Cost of frequent transmission is high
- Less frequent transmission implies that deadlocks may take longer to be detected

Distributed deadlock

B. Edge chasing

- Global wait-for graph is not built: servers send probes which contain the edges detected so far
  - Probe is sent if a transaction is waiting for a lock held by another transaction that is also waiting for a lock
  - Status of each transaction is obtained from its coordinator
- Deadlock cycles tend to be small → paths are short



Timestamp ordering

- The coordinator of a transaction assigns it a globally unique timestamp when it starts
  - As different servers may act as coordinators, they must agree on the order of the timestamps they generate, e.g. <local timestamp, server-id>
- Locally to each server, protocol acts normally

Optimistic concurrency control

- A transaction is validated by the collection of servers managing the objects it has accessed
- Different servers may serialize the same set of transactions in different orders
  A. Perform local validation and then check globally if the combination of the orderings is serializable
     - Transaction being validated is not involved in a cycle
  B. Coordinator assigns to the transaction a global sequence number that all participants must use
     - As different servers may act as coordinators, they must agree on the order of the numbers they generate

Summary

- Transactions group sequences of operations into an ACID operation
- Problem is how to increase concurrency while preserving serial equivalence
  - Two-phase locking
  - Timestamp ordering
  - Optimistic concurrency control
- Commit protocols provide the transactions with distributed atomicity
- Further details:
  - [Tanenbaum]: chapter 8.5
  - [Coulouris]: chapters 16 and 17

3.B. Consistency and Replication

Concurrence, Parallelism and Distributed Systems (CPDS)
Facultat d'Informàtica de Barcelona (FIB)
Universitat Politècnica de Catalunya (UPC)
2025/2026

FIB

Contents

- **Introducing replication & consistency**
- Data-centric consistency models
- Client-centric consistency models
- Consistency protocols

Introduction

- Reasons for replication
  A. Increase **availability**: data is available despite server failures and network partitions
  B. Enhance **reliability**: data is correct on the presence of faults (e.g. protection against data corruption, Byzantine failures, and stale data)
  C. Improve **performance**: this directly supports the goal of enhanced **scalability**
     - Size: Replicate data and distribute work instead of having one single server
     - Geographical: Replicate data close to where it is used (e.g. data caching)

Introduction

- What are the issues with replication?
  - Replication should be **transparent**
    - Clients are not aware that multiple copies of data exist
  - Replicated data should be **consistent**
    - Clients see the same data despite the replica they read

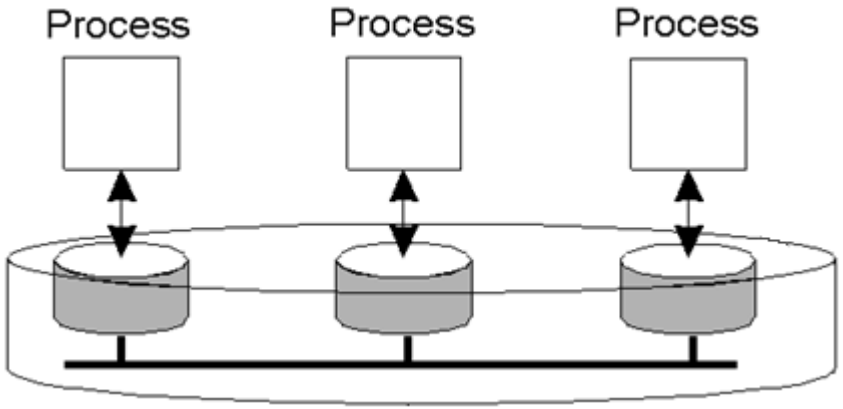
  ⇒ How do we transparently (and efficiently) keep all the replicas up-to-date and consistent?
- Dilemma: replication improves scalability, but incurs the overhead of synchronizing replicas
  - The solution often results in a relaxation of the consistency constraints

Contents

- Introducing replication & consistency
- **Data-centric consistency models**
- Client-centric consistency models
- Consistency protocols

Consistency models

- Distributed data store
  - Distributed across multiple nodes in a replicated fashion
  - Each replica holds a copy of the entire store
  - Each process can access one or several replicas
  - Write operations to a replica are propagated to the others



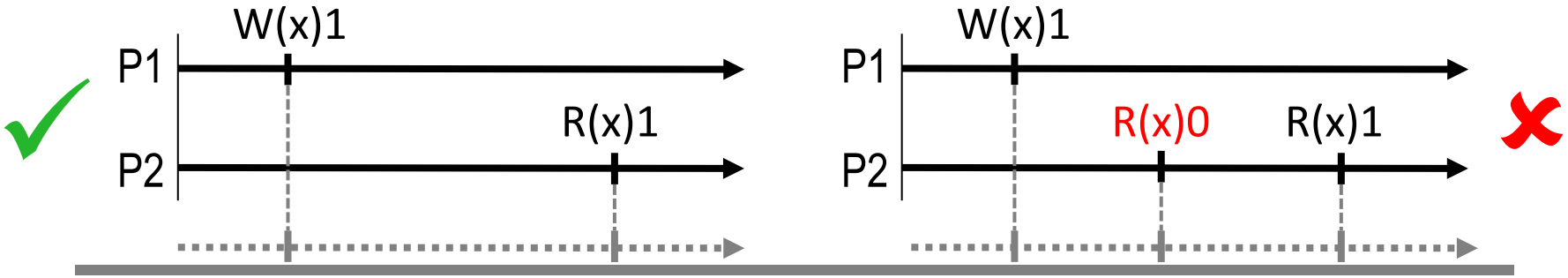
- A consistency model specifies precisely what the results of concurrent reads/writes are

Consistency models

- Diagram notation
- Time axis drawn horizontally with time increasing from left to right in all diagrams
- **$W_i(x)a$**: a write by process 'i' to item 'x' with a value 'a'
  - i.e. 'x' is set to 'a'
  - Note: The process is often shown as 'P_i'
- **$R_i(x)b$**: a read by process 'i' from item 'x' returning the value 'b'
  - i.e. reading 'x' returns 'b'

Strict consistency

- Meet sequential specification of each item 'x'
  - EXCLUSIVE ORDERED WRITES: Given 2 concurrent writes on 'x', one must go first
  - UP-TO-DATE MONOTONIC READS: A read on 'x' returns the value of the most recent write on 'x', regardless of the used replica
  - UP-TO-DATE MONOTONIC READS: Subsequent reads on 'x' return the same value or some later value, regardless of the used replica
- Operations run in a unique legal sequence that matches their **real-time** occurrence



Strict consistency

- Theoretical model that corresponds to full replication transparency
  - Also the model of uniprocessor systems
- Impossible to achieve in a distributed system
  1. Writes must be instantly visible in all replicas

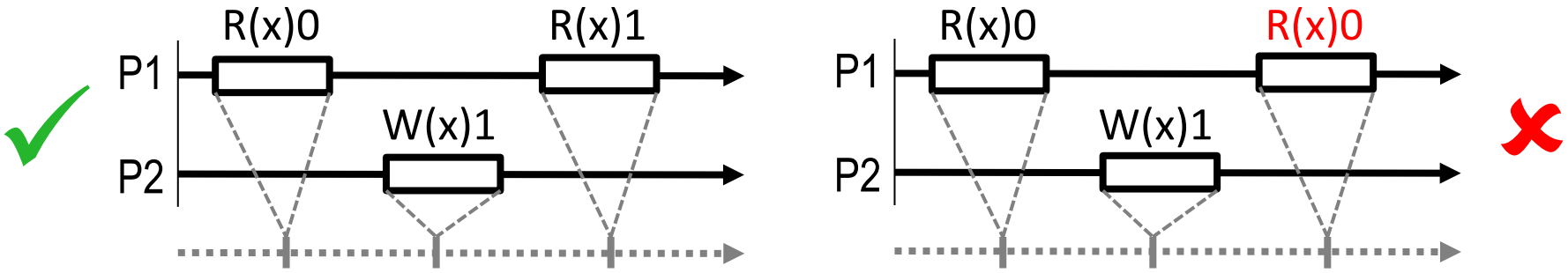
  ⇒ Network latencies preclude this

  2. Requires an absolute global time such that at most one operation occurs at a time

  ⇒ Clocks can neither be perfectly synchronized nor have infinite precision

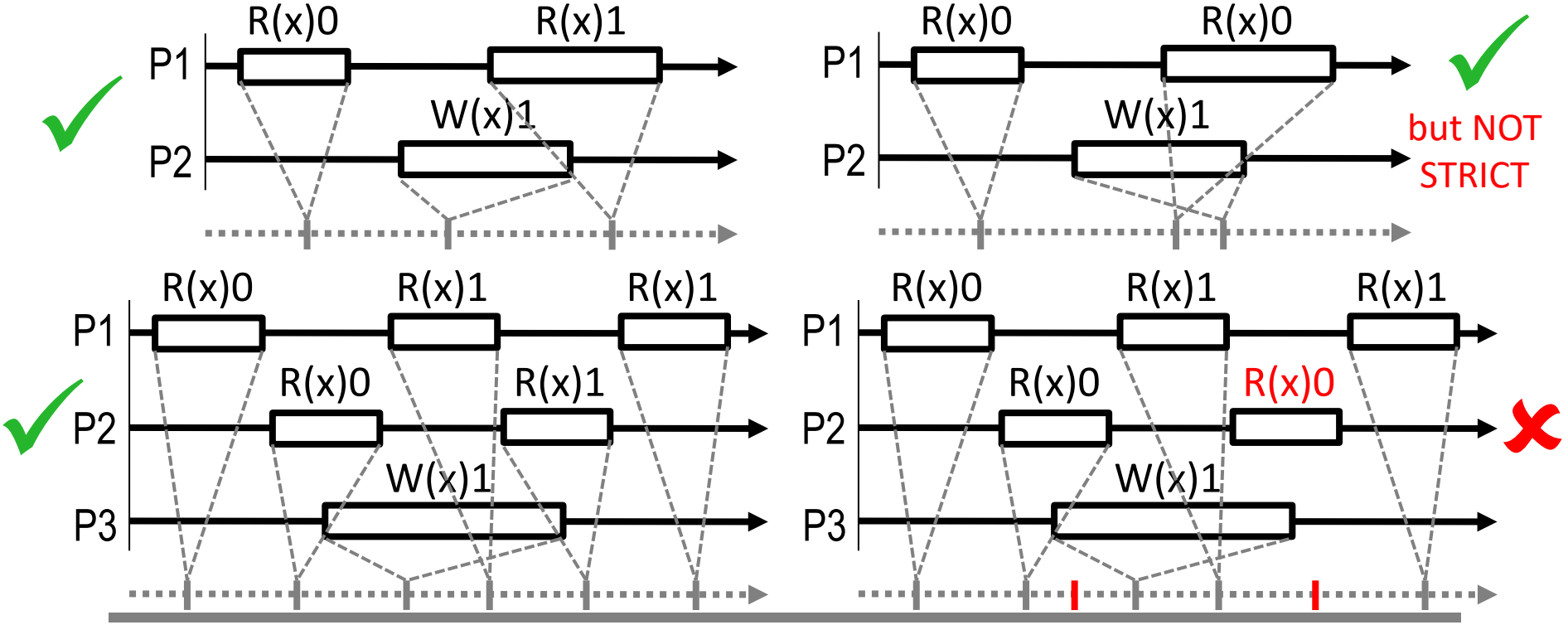
Linearizability (a.k.a. strong consistency)

- Same goal as strict consistency (exclusive ordered writes + up-to-date monotonic reads) but recognizing that operations take time to complete and can overlap
- Operations appear to run in some common sequence that retains their real-time order when they do not overlap



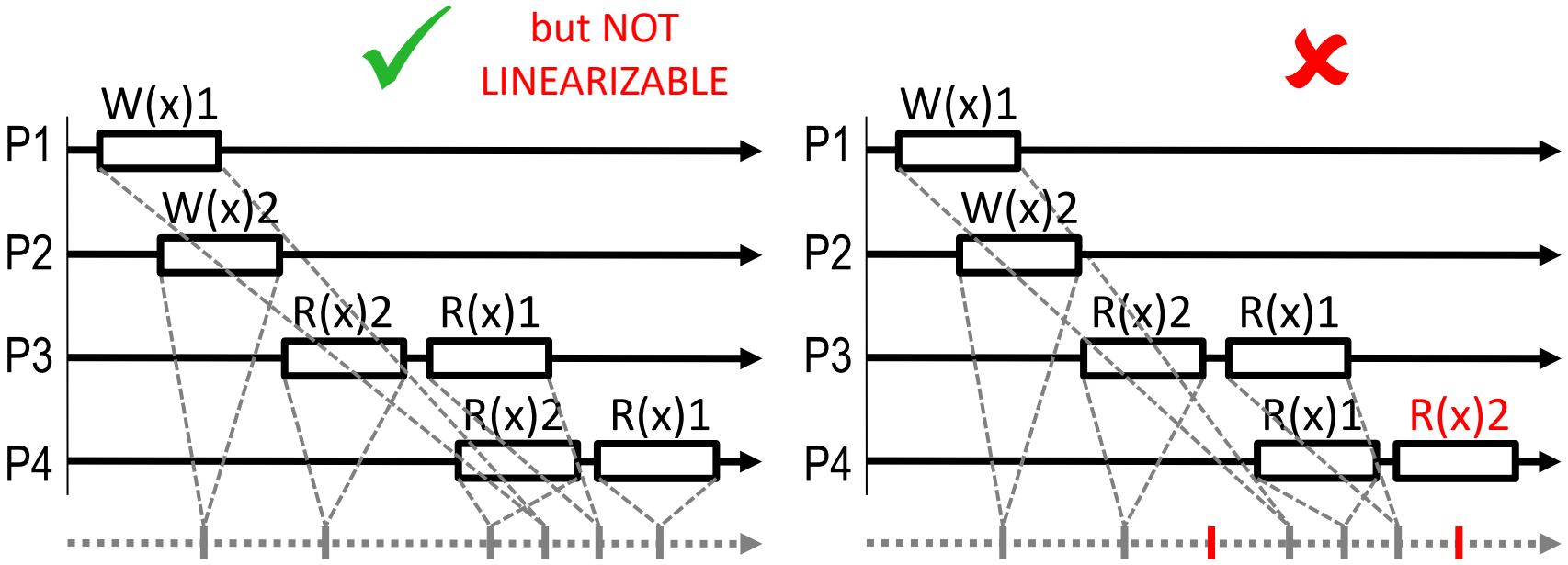
Linearizability (a.k.a. strong consistency)

- Overlapping ops. can take effect in any order
  - But their effect must become visible to everyone before they return (appearing to be atomic)



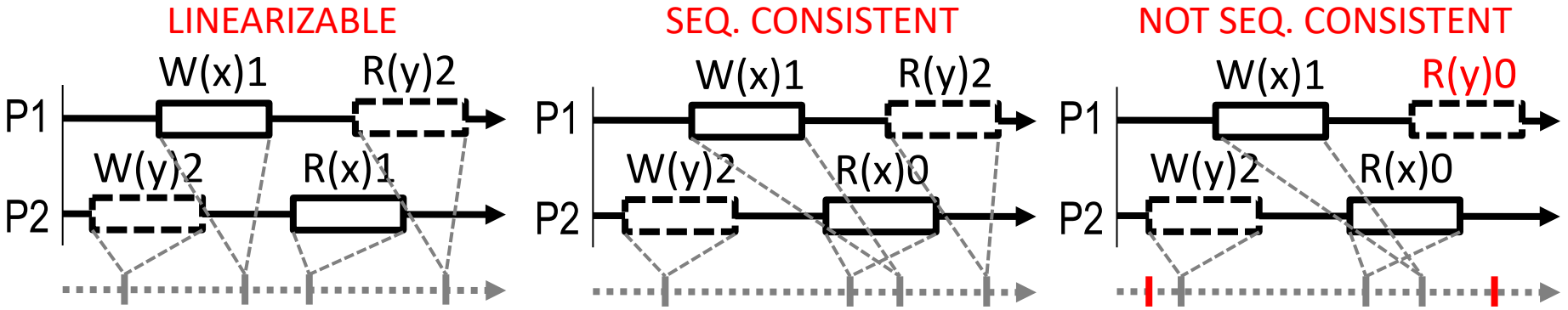
Sequential consistency

- Operations appear to run in some common sequence that retains the program order for the operations of each process
  - Real time is not taken into consideration



Linearizability vs. sequential consistency

- Only linearizability is compositional
  - Separated linearizable histories of objects can be composed in a linearizable history



- Both can be provided in distributed systems
  - Linearizability is stronger and more expensive
    - Strict ⊂ Linearizability ⊂ Sequential
  - See 'Consistency protocols' lesson for details

Contents

- Introducing replication & consistency
- Data-centric consistency models
- **Client-centric consistency models**
- Consistency protocols

Eventual consistency

- Data-centric models aim to provide a system-wide consistent view on data stores with concurrent write operations
  - These models require highly available connections
- **Eventual consistency** targets stores that:
  - Execute mainly read operations and have none or few write-write conflicts
  - Have users that often accept reading stale data
  - Deal with disconnected users, network partitions, and users preferring availability vs. consistency
  - e.g. DNS, Amazon Dynamo, Dropbox

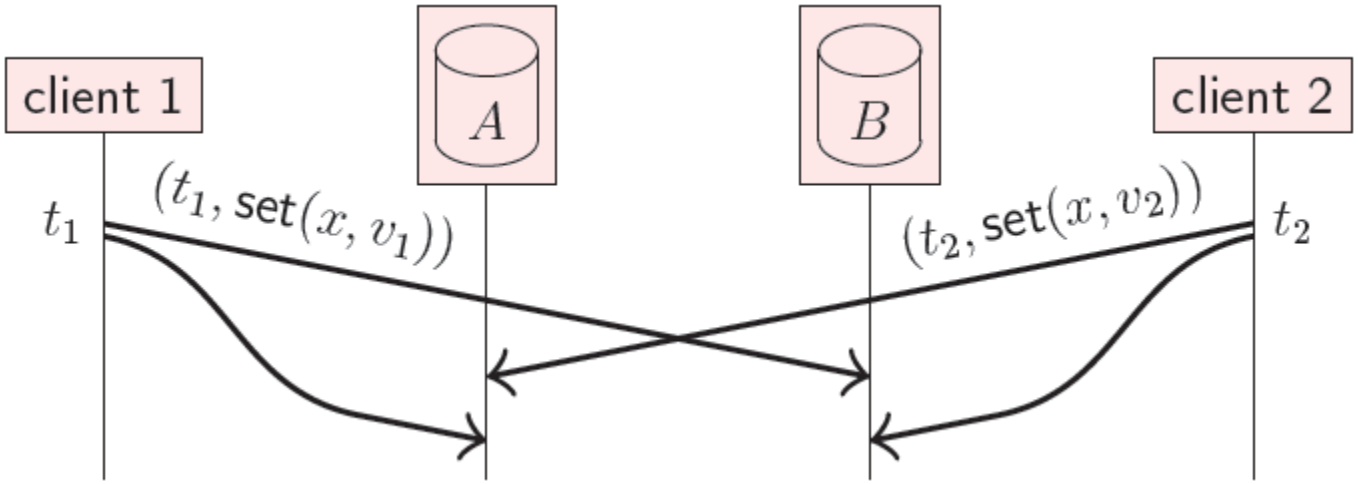
Eventual consistency

- Def: In absence of new updates, eventually all replicas will converge to identical values
- Each client operates on one or more replicas, updates propagate to others asynchronously, when connections are available
- Writes might arrive in different orders to the replicas: potential write-write conflicts
  - Replicas must apply the updates in the same order to ensure eventual convergence ⇒ this requires consistent conflict resolution for concurrent writes

Conflict resolution: Last writer wins

- Order writes according to clock timestamp
- Overwrite value only if timestamp of incoming write is higher than the current one

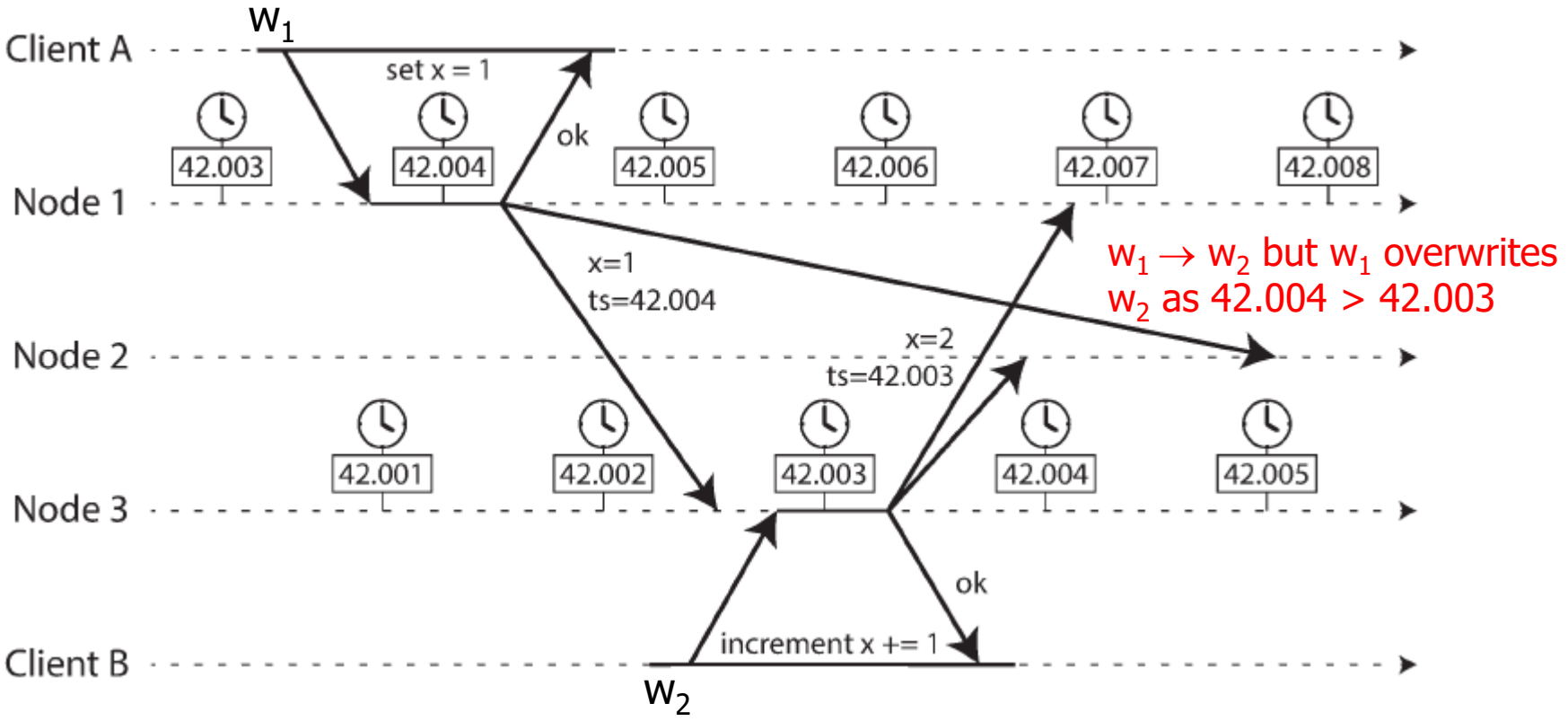
↓ Some concurrent writes are silently dropped



if t2 > t1 then A and B will keep v2 and discard v1

Conflict resolution: Last writer wins

↓ Later writes can be overwritten by happened-before writes (not concurrent) due to clock skew



Conflict resolution: Version vectors

- Each replica keeps a vector for each item with its version number per replica
  - Increment its own version number on every write
- Each incoming write includes the vector from a prior read (a.k.a. causal context)
- Replica compares both vectors to distinguish causally-related and concurrent writes
  - Overwrite values from happened-before writes
  - Keep values from concurrent writes as siblings
    - Siblings can be merged together by prompting the user or automatically (CRDTs, Operational Transformation)

Conflict resolution: Version vectors

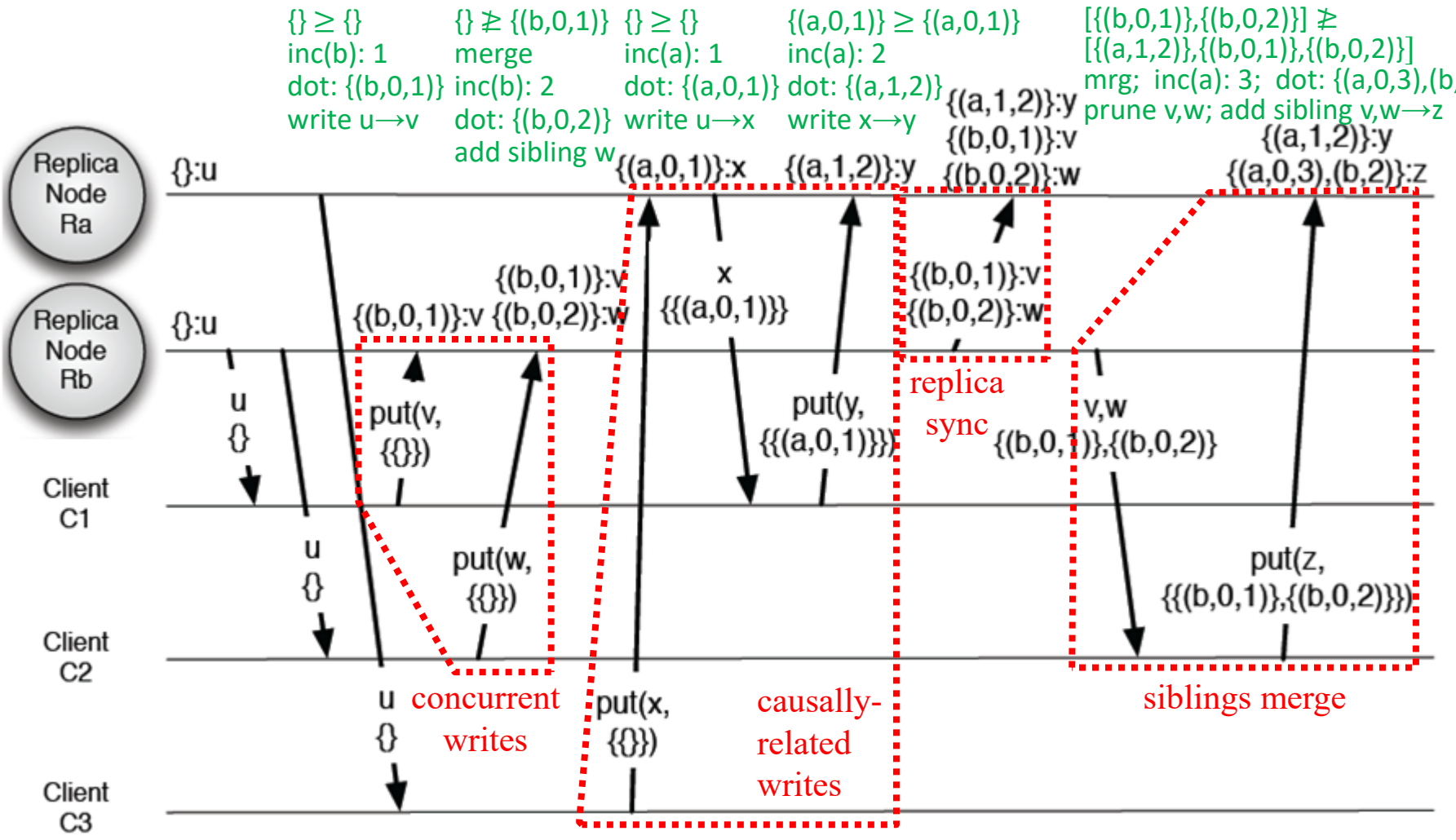
- Dotted version vectors implement this idea
  - Each replica k keeps a version vector $VV_k=\{(r_1, i_1, j_1), \ldots, (r_n, i_n, j_n)\}$ for each item
    - (replica r_k id, range of events $\{1, i_k\}$ [, last event j_k])
    - e.g. {(a, 2),(b, 1),(c, 2, 4)} ≡ {a1, a2, b1, c1, c2, c4}
  - Each incoming write includes the vector VV_{inc}

dot

if $VV_{inc} \geq VV_k$ **then**
increment VV_{inc}
get last event of VV_{inc} as dot
write incoming as sole value

otherwise
VV_{new} = merge VV_{inc} & VV_k (pairwise maximum)
increment VV_{new}
get last event of VV_{new} as dot
(use causal context from VV_{inc})
add new value as a **sibling**
prune all siblings s which $VV_{inc} \geq VV_k(s)$

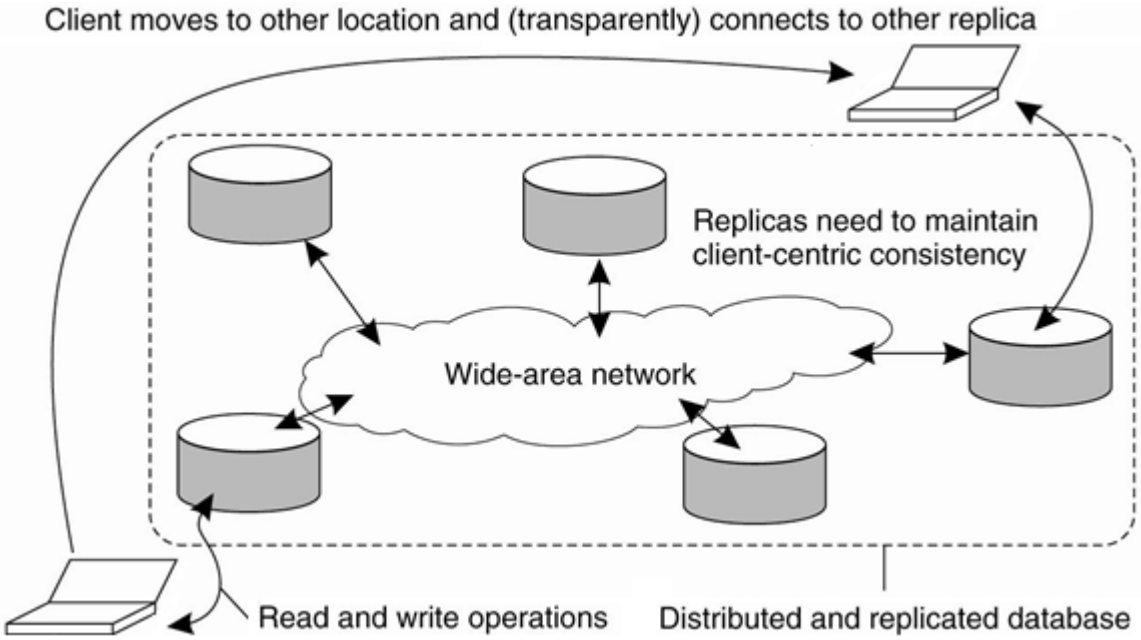
Conflict resolution: Version vectors



Eventual consistency

- Eventual consistency works well as long as every client always updates the same replica
- What if clients are mobile (operate on different replicas)?

⇒ Use **client-centric** consistency models (a.k.a. session guarantees)



Client-centric consistency models

- Guarantee that **a single client** sees its accesses to the data store in a consistent way
  - No guarantees are given concerning concurrent accesses by multiple clients
- 1) Read Your Writes, 2) Monotonic Reads, 3) Monotonic Writes, 4) Writes Follow Reads
  - Implemented by keeping track of version vectors representing two sets of writes for each client
    - **Read set** (RS_i): writes relevant (whose effects were visible) for the read operations performed by client i
    - **Write set** (WS_i): writes performed by client i

Client-centric consistency models

CLIENT i: C_i

val = read()

send(rd, RS_i, WS_i) to R_k

…

recv(val, VV_k) from R_k

RS_i= max(RS_i, VV_k)

return val

write(val)

send(wr, val, RS_i, WS_i) to R_k

…

recv(VV_k) from R_k

WS_i= max(WS_i, VV_k)

max: pairwise maximum

REPLICA k: R_k

read(RS_i, WS_i)

if MR then while $VV_k \not\geq RS_i$ {}

if RYW then while $VV_k \not\geq WS_i$ {}

val ← do read ; send(val, VV_k) to C_i

write(val, RS_i, WS_i)

if MW then while $VV_k \not\geq WS_i$ {}

if WFR then while $VV_k \not\geq RS_i$ {}

do write or add sibling†

increment VV_k ; send(VV_k) to C_i

foreach j≠k do send(up, val, VV_k) to R_j

update(val, VV_j)

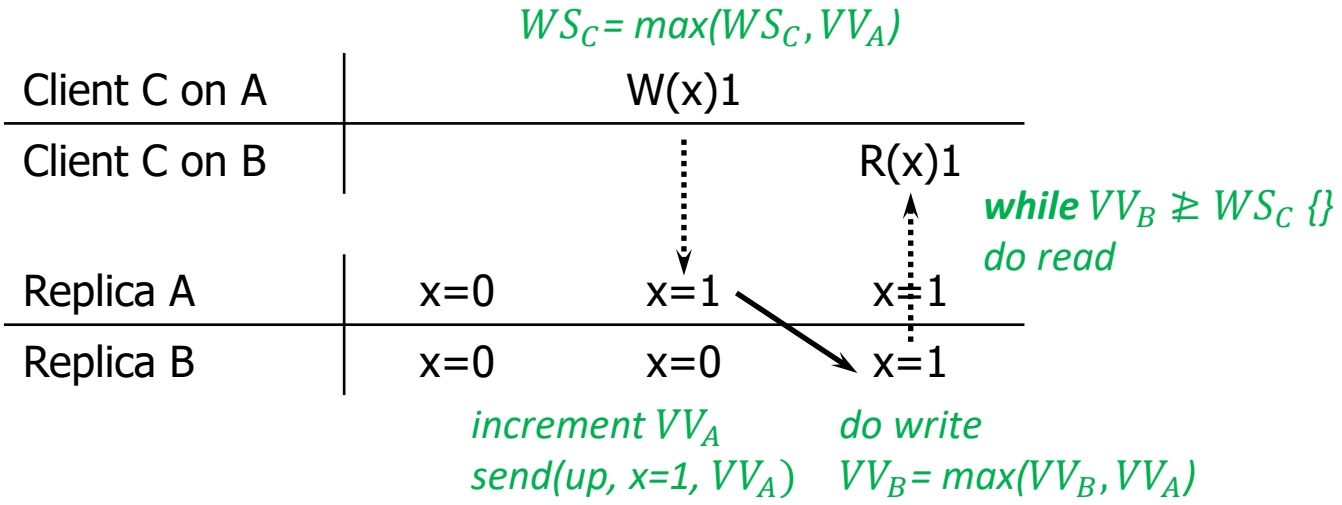
do write or add sibling†

VV_k= max(VV_j, VV_k)

† see 77-79 for details

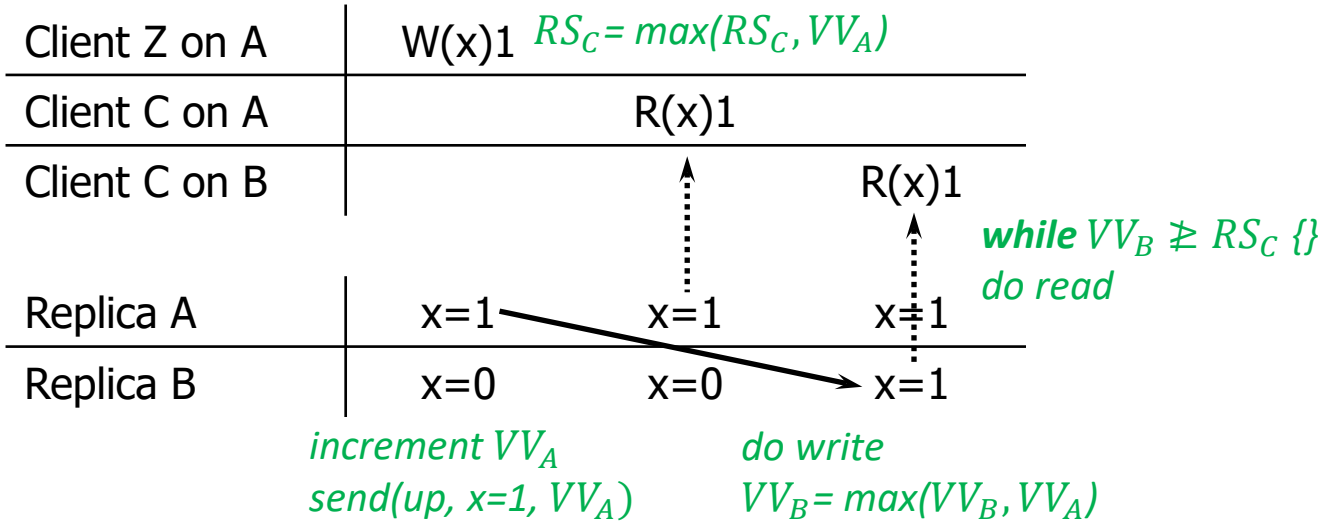
Read Your Writes (RYW)

- If client C writes a data item 'x' on replica A, any successive read on 'x' by C on replica B will return the written value or a more recent one
  - e.g. updating your web page



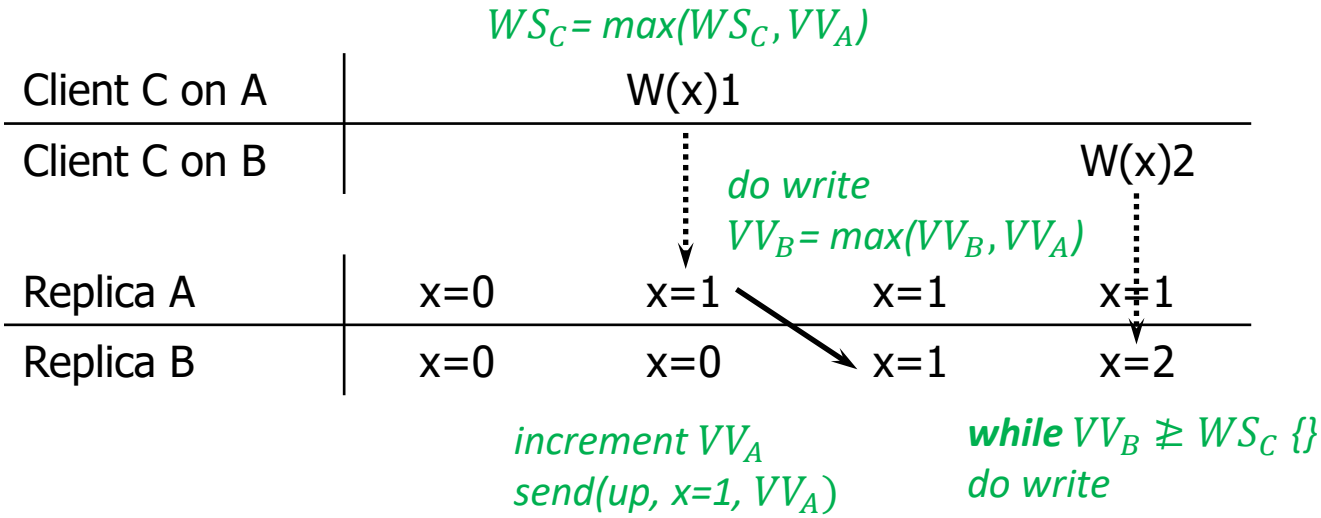
Monotonic Reads (MR)

- If client C reads a data item 'x' on replica A, any successive read on 'x' by C on replica B will return the same value or a more recent one
  - e.g. reading email while you are on the move



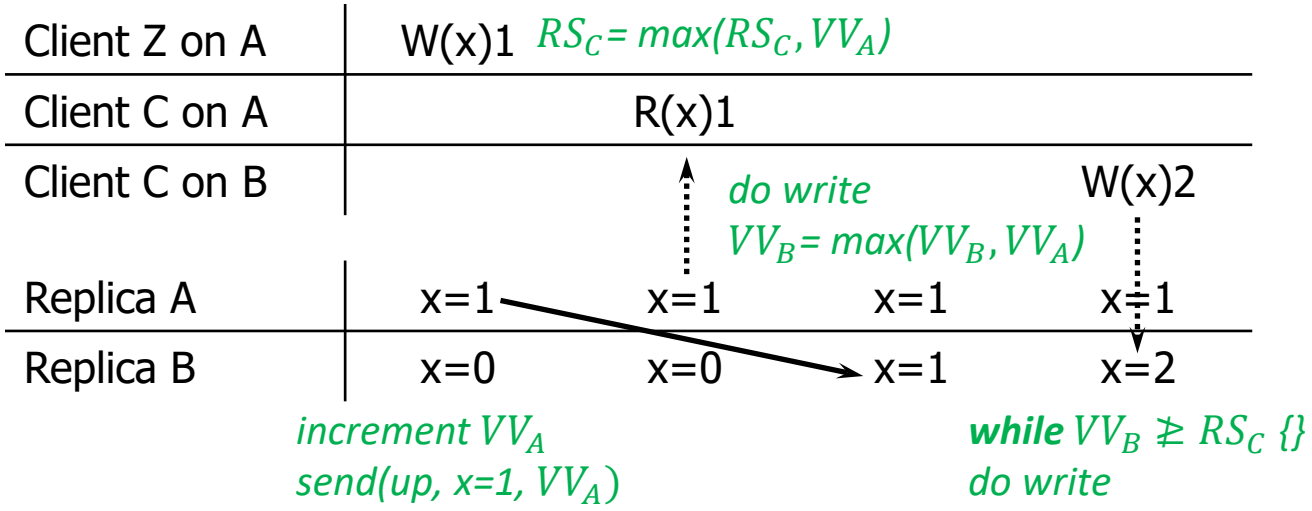
Monotonic Writes (MW)

- A write by client C on a data item 'x' on replica A is completed also in replica B before C performs any successive write on 'x' on replica B
  - e.g. versioning a software library



Writes Follow Reads (WFR)

- If C reads a data item 'x' on replica A, relevant writes for that read are completed also in replica B before C performs any successive write on 'x' on replica B
  - e.g. distributed newsgroup. A reaction (write) can be posted only after seeing the original post (read)



Consistency models on real systems

- Real systems supporting strong models:
  - NewSQL data stores: Google Spanner, Microsoft Yesquel
  - NoSQL data stores: Hyperdex, Apache HBase, MongoDB (*), COPS & Eiger (causal)
  - Distributed shared memory: IVY

(*) extended to support also eventual consistency

- Real systems supporting eventual models:
  - NoSQL data stores: Apache CoughDB, Amazon Dynamo (*), Apache Cassandra (*), Riak (*), LinkedIn Voldemort
  - File synchronizers: Dropbox

(*) extended to support also strong consistency

Contents

- Introducing replication & consistency
- Data-centric consistency models
- Client-centric consistency models
- **Consistency protocols**

Consistency protocols

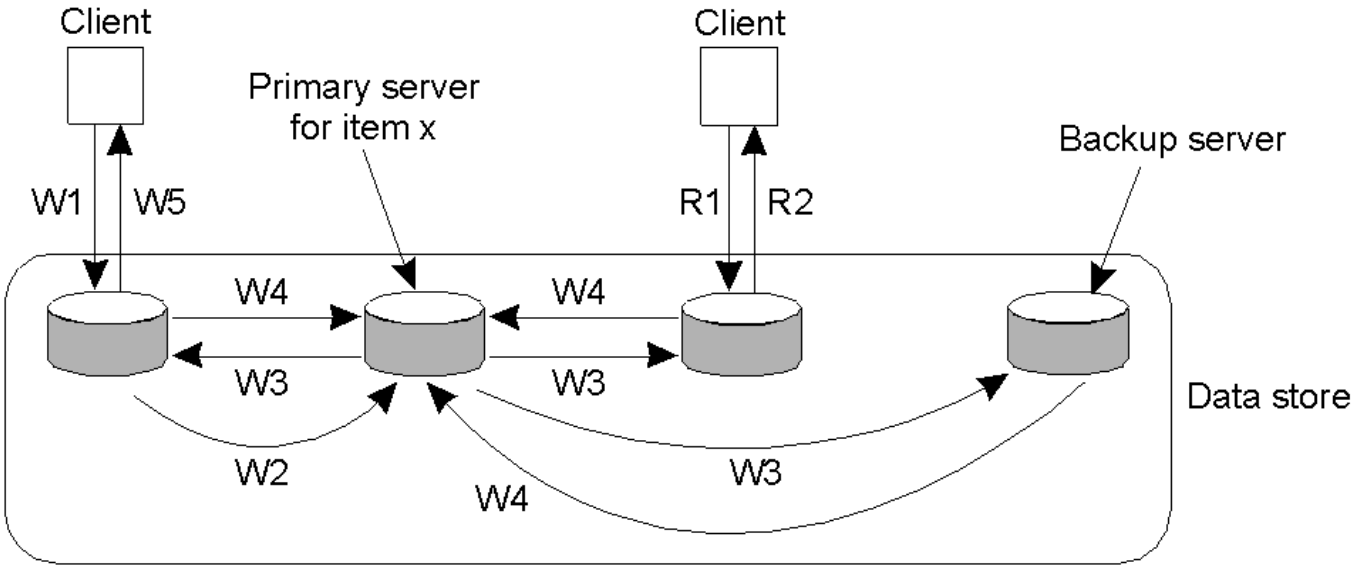
- Protocols that describe the implementation of specific consistency models
- In this lesson, we focus on sequential consistency and linearizability
- Consistency protocols discussed:
  1. Primary-based protocols
     - Writes are managed by a single **primary replica**
       - If it fails, another replica is elected to act as primary
  2. Replicated-write protocols
     - Writes are managed by **multiple replicas**

Contents

- Introducing replication & consistency
- Data-centric consistency models
- Client-centric consistency models
- **Consistency protocols**
  - **Primary-based protocols**
  - Replicated-write protocols

Primary-backup remote-write protocols

- All writes are forwarded to the primary replica
- Reads can be carried out locally



W1. Write request
W2. Forward request to primary
W3. Tell backups to update
W4. Acknowledge update
W5. Acknowledge write completed

R1. Read request
R2. Response to read

Primary-backup remote-write protocols

↓ Bad write performance

- Writes can take a long time because a blocking write protocol is used

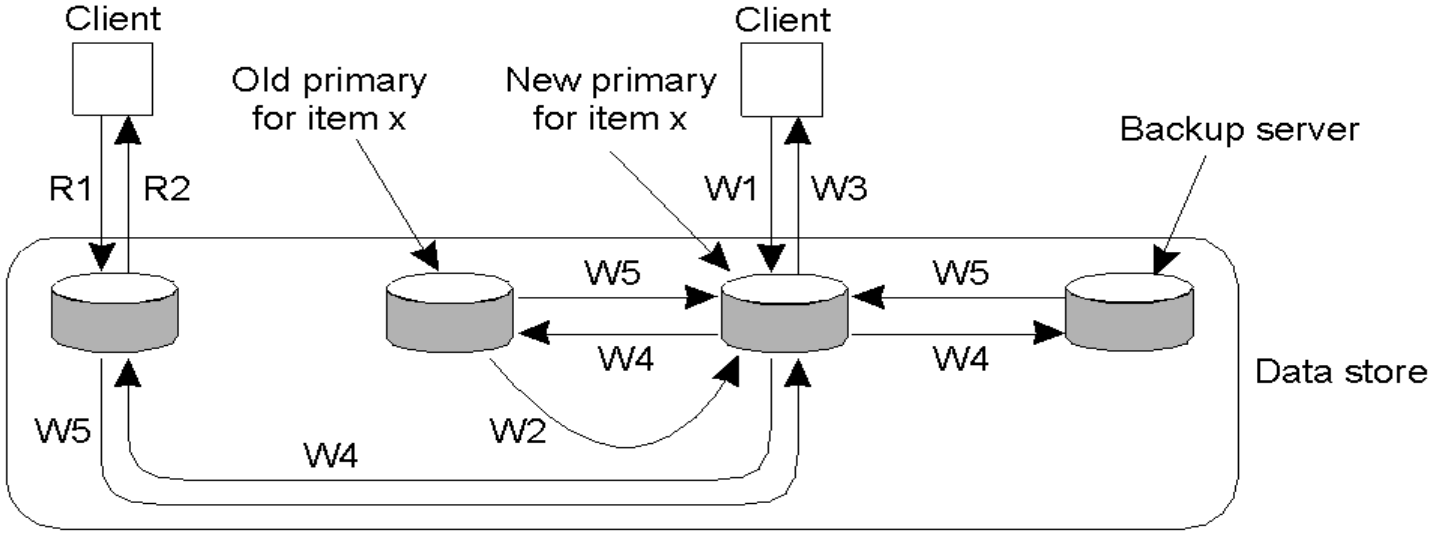
- Alternative: Use a non-blocking write protocol
  - Primary acknowledges the client's replica just after updating its local copy
  - If some replica failed during the update, read-your-writes consistency is not guaranteed

Primary-backup remote-write protocols

- ↑ Easy to implement sequential consistency
  - Primary sends the write operations to each replica via FIFO-ordered view-synchronous atomic multicast (see 'Multicast' lesson for details)
    - Having a single primary and FIFO-order ensure that all replicas will see the writes in the same order
    - Virtual synchrony allows the system to take over exactly where it left off upon primary failure
- Implements linearizability if read requests are also forwarded to the primary

Primary-backup local-write protocols

- Primary migrates to the replica that is writing, successive writes are carried out locally, and then the replicas are updated using a non-blocking protocol



W1. Write request
W2. Move item x to new primary
W3. Acknowledge write completed
W4. Tell backups to update
W5. Acknowledge update

R1. Read request
R2. Response to read

Contents

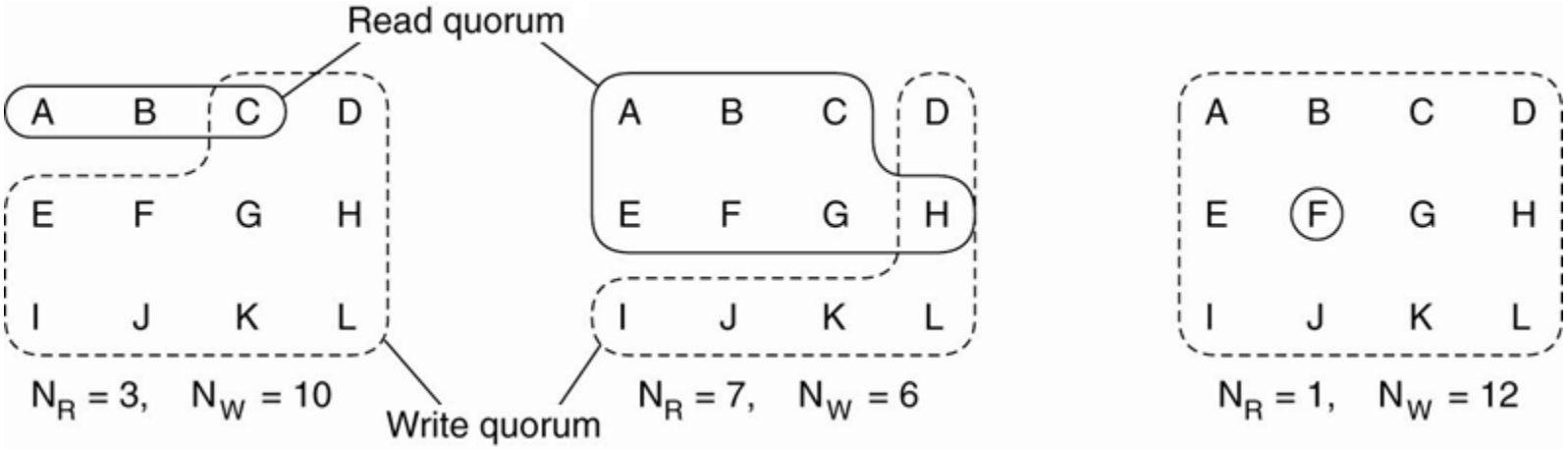
- Introducing replication & consistency
- Data-centric consistency models
- Client-centric consistency models
- **Consistency protocols**
  - Primary-based protocols
  - **Replicated-write protocols**

Active replication protocols

- Clients broadcast each operation to **all** the replicas and they are carried out in the same order everywhere
  - Use **total+FIFO-ordered multicast** to send the operations to the replicas
    - See 'Multicast' lesson for details
  - Sender waits until the multicast message is delivered back and then it returns to the client
- Multicasting reads+writes → linearizability
- Multicasting only writes → sequential cons.
  - Reads return the current value of the local copy

Quorum-based protocols

- Clients must get replies from a quorum of the N replicas before reading (N_R) or writing (N_W)
  - **$N_R + N_W > N$** to avoid read-write conflicts
  - **$N_W > N/2$** to avoid write-write conflicts



correct choice of read and write set

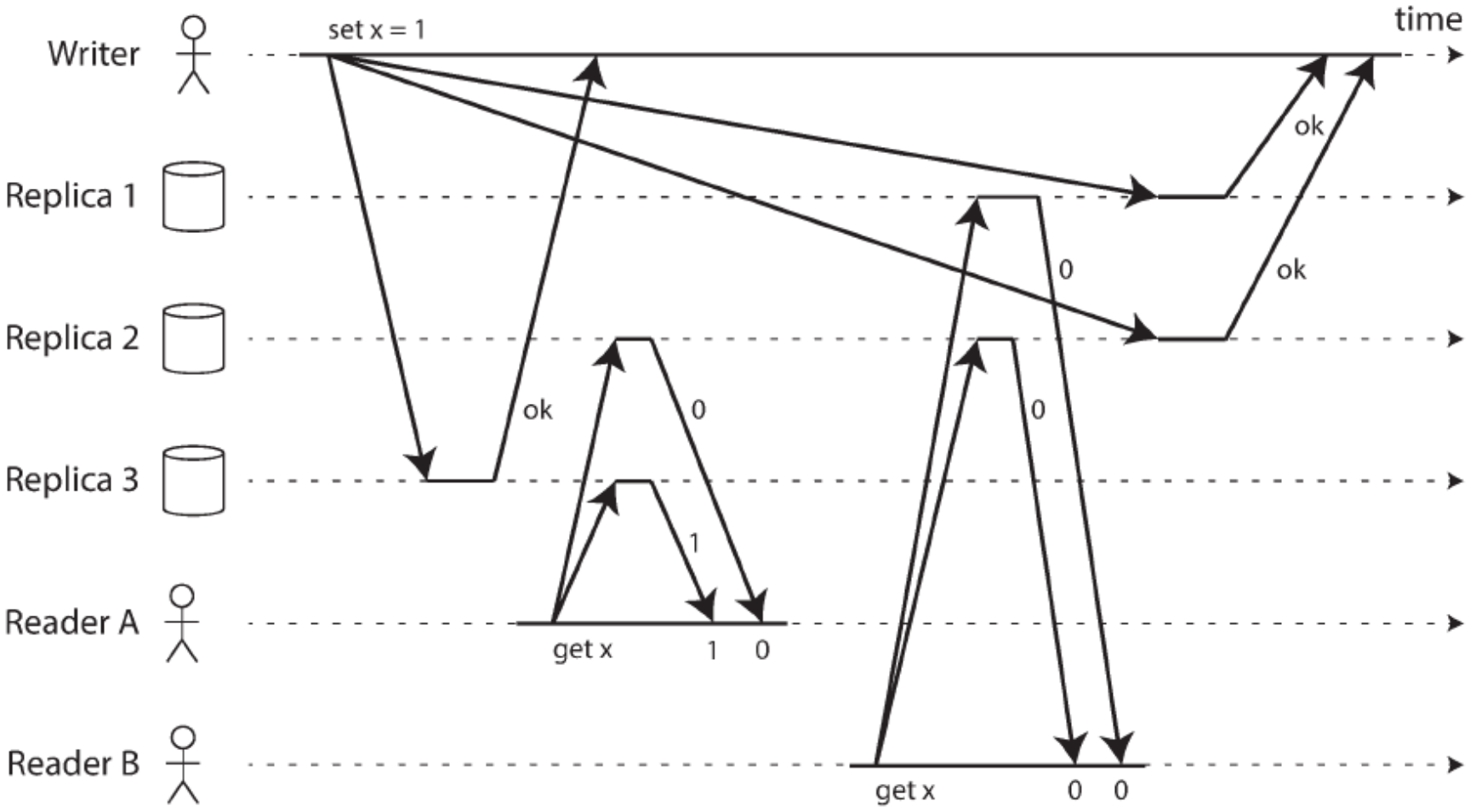
choice that may lead to WR-WR conflicts

correct choice: ROWA (read-one, write-all)

Quorum-based protocols

- These protocols can implement linearizability, but this requires additional steps in addition to reading/writing from a quorum
  1. Use majority quorums for reading and writing
     - ($N_R > N/2$) and ($N_W > N/2$)
  2. Use version vectors to achieve ordered writes according to their real-time order
     - Writes must read first to learn the highest version and the potential siblings ⇒ see 'Conflict resolution' lesson
  3. Perform 'read repair' to achieve up-to-date monotonic reads
     - Read operations must write to update stale replicas

Why we need 'read repair'



Quorum write

1. Read phase
   - Send read requests to all (or N_R) replicas
   - Await replies from N_R replicas
   - Learn the highest version among the replicas
     - Merge siblings if needed
2. Write phase
   - Send write requests (containing the version learnt) to all (or N_W) replicas (if N_W, remaining replicas are updated as a background task: anti-entropy)
   - Await ACKs from N_W replicas
   - Write is complete and can return to the user

Quorum read

1. Read phase
   - Send read requests to all (or N_R) replicas
   - Await replies from N_R replicas
   - If all version numbers are the same, return the value to the user
2. Write phase ('read repair')
   - Otherwise, send write requests (containing the largest version number and the associated up-to-date value) to all (or only outdated) replicas
   - Await ACKs from N_W (or only updated) replicas
   - Return the value to the user

Summary

- Reasons for replication
  - Improved availability
  - Enhanced reliability
  - Improved performance & scalability
- But replication can lead to inconsistencies …
- How can we propagate updates so that these inconsistencies are not noticed without severely degrading performance?
- Proposed solutions apply for the relaxation of any existing consistency constraints

Summary

- Consistency models
  - Data-centric models
    - Strict, Linearizability, Sequential provide a system-wide consistent view on data stores with concurrent individual reads/writes to data items
  - Client-centric models
    - Concerned with maintaining consistency for the access of a single client to the distributed data store
    - Models based on Eventual consistency
      - Read your writes, Monotonic reads, Monotonic writes, Writes follow reads

Summary

- We looked at 'consistency protocols' as a way to implement consistency models
  - The most common schemes are those that support sequential consistency or linearizability, but eventual consistency + sessions guarantees is gaining popularity
- Further details:
  - [Tanenbaum]: chapter 7
  - [Coulouris]: chapter 18